

SEND5232: Software Architecture and Design Project

Bereket Kiros – ugr/189518/16 – Section 2



Phase 7 — Quality Evaluation



Lihq'et Bookstore System

1. Introduction

This section evaluates the architectural design of the Lihq'et Bookstore System based on key software quality attributes. The evaluation focuses on how the chosen architecture (3-tier layered architecture combined with MVC and Service Layer patterns) supports system goals such as scalability, performance, maintainability, security, and usability.

2. Scalability Evaluation

The system is designed with a modular layered architecture, which improves scalability. The separation between Presentation, Application, and Data layers allows each layer to evolve independently.

- The backend can be scaled horizontally by adding more Node.js instances.
- The Service Layer allows new business features (e.g., authentication, search, analytics) to be added without modifying Controllers or Models extensively.
- MySQL can be scaled using replication or migration to cloud-based databases in future enhancements.

Overall, the system supports **moderate scalability suitable for small-to-medium applications**.

3. Performance Evaluation

Performance is addressed through lightweight backend design and efficient data handling:

- Node.js uses a non-blocking I/O model, enabling efficient handling of concurrent requests.
- Direct and optimized SQL queries improve database response time.
- Separation of business logic into the Service Layer reduces controller complexity and improves request processing efficiency.

However, caching mechanisms (e.g., Redis) are not implemented in this version, which could further enhance performance in future iterations.

4. Maintainability Evaluation

The system achieves high maintainability due to:

- **MVC Architecture:** Separates responsibilities into Controller, Service, and Model layers.
- **Service Layer Pattern:** Centralizes business logic, making updates easier and reducing code duplication.
- Modular frontend and backend separation.

This structure ensures that changes in one layer do not heavily impact others, improving long-term maintainability.

5. Security Evaluation

Security is moderately addressed in the current design:

- Input validation is handled in the Service Layer before database operations.
- SQL queries are controlled through the Model layer, reducing direct exposure to database operations.
- RESTful API structure ensures controlled communication between frontend and backend.

However, advanced security features such as authentication, authorization, and encryption are not implemented and are identified as future enhancements.

6. Usability Evaluation

The system is designed with simplicity and ease of use in mind:

- Clean and minimal user interface for book management.
- Intuitive CRUD operations (Add, View, Edit, Delete).
- Immediate UI updates without page reloads improve user experience.

The usability is suitable for non-technical users such as bookstore staff.

7. Reliability and Availability

The system demonstrates basic reliability:

- Database persistence ensures data is not lost on refresh or restart.
- Error handling in API requests ensures graceful failure responses.
- Separation of concerns reduces system-wide failure risk.

However, high availability mechanisms (such as load balancing or redundancy) are not implemented in this prototype.

8. Architectural Justification Summary

The chosen architecture (Layered + MVC + Service Layer) was selected because it provides:

- Clear separation of concerns
- Easy scalability for future features
- Maintainable and modular code structure
- Simplified debugging and development process

This makes the system suitable as a foundation for future expansion into a full-featured bookstore or inventory management system.

9. Conclusion

The Lihq'et Bookstore System architecture successfully satisfies core software quality attributes at a foundational level. While it remains a prototype system, it demonstrates strong principles of modularity, maintainability, and structured design. Future improvements such as authentication, caching, and deployment scaling can further enhance its production readiness.